

# SMART HOSTEL MANAGEMENT SYSTEM

## A Java-Based Object-Oriented Programming Project

---

### 1. Abstract

The **Smart Hostel Management System** is a Java-based console application developed to simplify and organize basic hostel management activities. The system provides an easy way to register students, manage rooms, allocate and vacate rooms, handle complaints, and calculate hostel fees.

The project is designed using important **Object-Oriented Programming (OOP)** concepts such as encapsulation, inheritance, polymorphism, abstraction, classes, objects, constructors, interfaces, and method overriding. ArrayList is used to manage multiple students, rooms, and complaints dynamically.

The application also implements **file handling** to store student, room, and complaint information in text files. This allows the information to be loaded again when the application is restarted. Exception handling is included to manage invalid user inputs and file-related errors.

The main objective of this project is to demonstrate how Java programming concepts can be applied to a practical real-world problem. The system provides a simple and organized approach to hostel administration while keeping the implementation understandable and maintainable.

---

### 2. Introduction

Hostel management involves maintaining information about students, rooms, room occupancy, complaints, and fees. Managing these activities manually can become difficult when the number of students and rooms increases.

The **Smart Hostel Management System** provides a basic computerized solution for these activities. It is a menu-driven Java application where the user can select different operations according to their requirements.

The system maintains student information such as student ID, name, phone number, course, and allocated room. It also manages different room types, including single and shared rooms. The system keeps track of room capacity and occupancy to prevent allocation when a room is full.

Another important feature is complaint management. Students can submit maintenance or electrical complaints, and the complaints can later be marked as resolved.

The project also demonstrates persistent data storage using text files. Student, room, and complaint information can be saved and loaded between program executions.

---

### **3. Problem Statement**

Traditional hostel management may involve maintaining student records, room allocation details, complaints, and fee information manually. This can lead to difficulty in finding information, maintaining accurate room occupancy, and preserving records.

The proposed system aims to provide a simple Java-based solution that can manage these activities in an organized manner while demonstrating fundamental OOP concepts.

---

### **4. Objectives**

The main objectives of the project are:

1. To create a simple hostel management application using Java.
  2. To register and manage student information.
  3. To add and manage different types of hostel rooms.
  4. To allocate rooms to registered students.
  5. To allow students to vacate their allocated rooms.
  6. To track room capacity and occupancy.
  7. To manage maintenance and electrical complaints.
  8. To calculate hostel fees based on room type.
  9. To save and load information using file handling.
  10. To demonstrate important Java OOP concepts in a practical project.
- 

### **5. Scope of the Project**

The project focuses on the basic operations required for hostel administration.

The system can be used to:

- Maintain student records.
- Maintain room records.
- Track available beds.
- Allocate rooms to students.
- Vacate rooms.
- Register hostel complaints.
- Resolve complaints.
- Calculate monthly hostel fees.
- Store information in text files.

The current version is designed as a console-based academic project. It can later be expanded into a GUI or database-based application.

---

## 6. Technologies Used

| Technology         | Purpose                                 |
|--------------------|-----------------------------------------|
| Java               | Main programming language               |
| Visual Studio Code | Development environment                 |
| Java OOP           | Application structure                   |
| ArrayList          | Dynamic data storage                    |
| Text Files         | Data persistence                        |
| File Handling      | Saving and loading data                 |
| Exception Handling | Handling invalid inputs and file errors |

---

## 7. System Requirements

### Hardware Requirements

- Computer or laptop
- Minimum 4 GB RAM
- Basic storage space
- Keyboard and monitor

### **Software Requirements**

- Java Development Kit (JDK)
  - Visual Studio Code or any Java-supported IDE
  - Windows/Linux/macOS operating system
  - Java compiler and runtime environment
- 

## **8. System Features**

### **8.1 Student Registration**

The user can register a student by entering:

- Student ID
- Name
- Phone number
- Course

The system checks whether the student ID already exists before adding the student.

### **8.2 Room Management**

The application supports:

- Single rooms
- Shared rooms

The user can specify the capacity of a shared room. The system keeps track of occupied and available beds.

### **8.3 Room Allocation**

A registered student can be assigned an available room. The system checks:

- Whether the student exists.
- Whether the room exists.
- Whether the student already has a room.
- Whether the room has an available bed.

#### 8.4 Room Vacation

When a student vacates a room, the student's room number is removed and the room's occupancy is reduced.

#### 8.5 Complaint Management

Students can submit:

- Maintenance complaints
- Electrical complaints

Complaints initially have a **Pending** status and can later be marked as **Resolved**.

#### 8.6 Hostel Fee Calculation

The system calculates the monthly fee according to the room type.

- Single Room: Rs. 8000
- Shared Room: Rs. 5000

#### 8.7 File Handling

The application saves information in:

data/

└─ students.txt

└─ rooms.txt

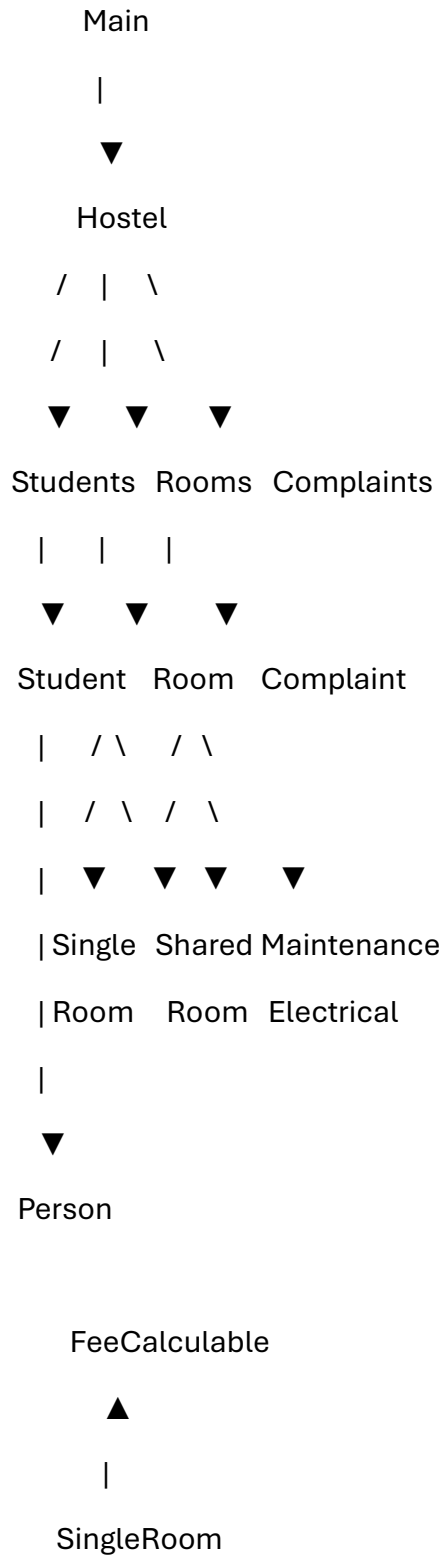
└─ complaints.txt

The saved information can be loaded when the application starts again.

---

### 9. System Architecture

The project is divided into multiple classes, with each class responsible for a particular part of the system.



SharedRoom

FileManager

|



Text Files

---

## 10. Class Description

### 10.1 Person

The Person class stores common personal information such as:

- ID
- Name
- Phone number

It acts as the parent class for Student.

### 10.2 Student

The Student class extends Person. It adds:

- Course
- Room number

It also overrides the displayDetails() method.

### 10.3 Room

The Room class contains common room information:

- Room number
- Capacity
- Occupied beds

It also provides methods for checking availability and adding or removing students.

### 10.4 SingleRoom

SingleRoom extends Room and represents a room with a capacity of one student.

### **10.5 SharedRoom**

SharedRoom extends Room and allows a user-defined room capacity.

### **10.6 Complaint**

The Complaint class stores:

- Complaint ID
- Student ID
- Description
- Status

The default status is "Pending".

### **10.7 MaintenanceComplaint**

This class extends Complaint and represents maintenance-related complaints.

### **10.8 ElectricalComplaint**

This class extends Complaint and represents electrical complaints.

### **10.9 Hostel**

The Hostel class acts as the main management class. It maintains lists of:

- Students
- Rooms
- Complaints

It also handles registration, allocation, complaints, fees, and file operations.

### **10.10 FileManager**

The FileManager class is responsible for saving and reading information from text files.

### **10.11 InputHelper**

The InputHelper class handles user input and prevents invalid integer input from crashing the application.

### **10.12 FeeCalculable**



FeeCalculable is an interface containing the calculateFee() method. Room classes implement this interface to calculate their respective fees.

---

## **11. Object-Oriented Programming Concepts**

### **11.1 Encapsulation**

Encapsulation is implemented by declaring data members as private.

For example:

```
private int id;  
  
private String name;  
  
private String phone;
```

Access is provided through methods such as getters and setters.

---

### **11.2 Inheritance**

Inheritance allows one class to acquire properties and methods from another class.

Example:

```
public class Student extends Person
```

Similarly:

```
public class SingleRoom extends Room
```

and:

```
public class SharedRoom extends Room
```

This reduces code duplication and represents relationships between classes.

---

### **11.3 Polymorphism**

Polymorphism is demonstrated through method overriding.

For example, displayRoom() is overridden by both SingleRoom and SharedRoom.

Similarly, complaint subclasses override displayComplaint().

---

## 11.4 Abstraction

Abstraction is demonstrated using the FeeCalculable interface.

```
public interface FeeCalculable {  
    double calculateFee();  
}
```

The individual room classes provide their own implementation of the method.

---

## 11.5 Constructors

Constructors are used to initialize objects when they are created.

For example:

```
Student student = new Student(  
    id,  
    name,  
    phone,  
    course  
);
```

---

## 12. Data Structures Used

The project uses ArrayList for dynamic storage.

```
private ArrayList<Student> students;  
  
private ArrayList<Room> rooms;  
  
private ArrayList<Complaint> complaints;
```

ArrayList was selected because the number of students, rooms, and complaints can change during program execution.

---

### 13. Exception Handling

Exception handling is used to prevent the program from terminating unexpectedly when invalid data is entered.

For example, InputHelper uses a try-catch block when accepting integer input.

```
try {  
    return scanner.nextInt();  
}  
  
catch (Exception e) {  
    System.out.println("Please enter a valid number.");  
    scanner.nextLine();  
}
```

File operations also use exception handling to handle IOException.

---

### 14. File Handling

The project uses text files for persistent storage.

The FileManager class provides methods to:

- Create the data folder.
- Save information.
- Read information.

Student information is stored in:

students.txt

Room information is stored in:

rooms.txt

Complaint information is stored in:

complaints.txt

This allows information to remain available after the application is closed.

---

## 15. Working of the System

The general working process is:

Start

|



Load saved data

|



Display main menu

|



Select operation

|

├— Register Student

├— Add Room

├— Allocate Room

├— Vacate Room

├— View Students

├— View Rooms

├— Submit Complaint

├— View Complaints

├— Resolve Complaint

├— Calculate Fee

└— Save Data

|



Return to Menu

|



Exit

---

## 16. Testing

The application was tested using different operations and inputs.

| Test Case                     | Expected Result            | Status |
|-------------------------------|----------------------------|--------|
| Register a new student        | Student added successfully | Pass   |
| Register duplicate student ID | Duplicate ID rejected      | Pass   |
| Add a room                    | Room added successfully    | Pass   |
| Add duplicate room            | Duplicate room rejected    | Pass   |
| Allocate available room       | Room allocated             | Pass   |
| Allocate full room            | Allocation rejected        | Pass   |
| Vacate allocated room         | Room becomes available     | Pass   |
| Submit complaint              | Complaint added            | Pass   |
| Resolve complaint             | Status changes to Resolved | Pass   |
| Calculate room fee            | Correct fee displayed      | Pass   |
| Save data                     | Data stored in text files  | Pass   |
| Restart application           | Saved data loaded          | Pass   |
| Enter invalid numeric input   | Error handled              | Pass   |

---

## 17. Sample Output

```
=====
```

## SMART HOSTEL MANAGEMENT

```
=====
```

1. Register Student
2. Add Room
3. Allocate Room
4. Vacate Room
5. View Students
6. View Rooms
7. Submit Complaint
8. View Complaints
9. Resolve Complaint
10. Calculate Hostel Fee
11. Save Data
0. Exit

```
=====
```

Enter your choice:

Example fee output:

===== HOSTEL FEE =====

Enter Student ID: 101

Student: Pratibha

Room Number: 201

Monthly Hostel Fee: Rs. 5000.0

---

## 18. Advantages

- Simple and easy-to-use interface
  - Reduces manual record management
  - Prevents allocation of full rooms
  - Maintains complaint status
  - Supports different room types
  - Provides automatic fee calculation
  - Stores data for future use
  - Demonstrates reusable OOP design
- 

## **19. Limitations**

The current version has some limitations:

- It is a console-based application.
- Data is stored in text files instead of a database.
- There is no login or authentication system.
- It does not provide an online payment system.
- It does not have a graphical user interface.
- Complaint categories are currently limited.

These limitations provide opportunities for future development.

---

## **20. Future Scope**

The project can be further improved by adding:

1. A graphical user interface using Java Swing or JavaFX.
2. Database connectivity using MySQL.
3. Student and administrator login systems.
4. Online hostel fee payment.
5. More detailed complaint categories.

6. Automatic receipt generation.
  7. Room-wise student information.
  8. Search and filtering functionality.
  9. Email or notification support.
  10. An administrator dashboard with hostel statistics.
- 

## 21. Conclusion

The **Smart Hostel Management System** successfully demonstrates how Java and Object-Oriented Programming concepts can be used to develop a practical application. The system provides essential hostel management features such as student registration, room management, room allocation, complaint handling, and fee calculation.

The project helped demonstrate the practical use of encapsulation, inheritance, polymorphism, abstraction, interfaces, constructors, collections, exception handling, and file handling. The use of separate classes makes the application organized and easier to maintain.

Although the current system is a console-based academic project, its structure provides a foundation for future improvements such as database integration, graphical interfaces, authentication, and online services.

Overall, the project provides practical experience in applying Java programming concepts to a real-world management problem.

---

## 22. References

1. Oracle Java Documentation — Java Programming Language.
  2. Java Platform, Standard Edition Documentation.
  3. Course material for Programming in Java.
  4. Visual Studio Code Documentation.
  5. Java documentation for ArrayList, exception handling, and file handling.
- 

## Project Information



**Project Title:** Smart Hostel Management System

**Programming Language:** Java

**Project Type:** Object-Oriented Programming Project

**Application Type:** Console-Based Application

**Data Storage:** Text Files

**Development Environment:** Visual Studio Code